

Jenkins and Pipeline

Qingsong Zhang, Ph. D.

9/24/2025

Core Concepts

- Jenkins **Controller** (Formerly Master)

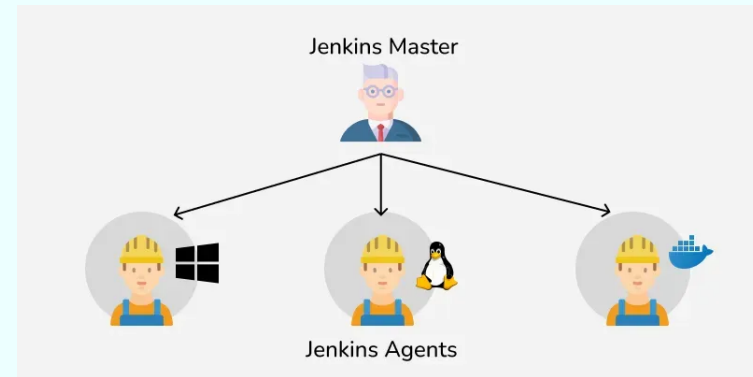
The Jenkins Controller serves as the central system for managing a Jenkins instance, often referred to as its "heart." It oversees agents and their connections, determining the tasks they should perform. Additionally, the Jenkins Controller loads plugins and ensures that jobs run in the correct sequence.

- Jenkins **Agent** (Formerly Slave)

Jenkins Agent is a machine that performs tasks like running scripts, executing tests, or building components, etc. These tasks are assigned by the Jenkins Controller. Each agent can have its setup, like different operating systems, software, or hardware. This helps Jenkins handle many types of tasks and work faster by spreading the load.

There are two main types of agents:

- **Permanent Agents:** These are always ready and connected to Jenkins. They're like dedicated workers who are always on standby.
- **Ephemeral Agents:** These are temporary. Jenkins starts them only when needed, usually in the cloud or using tools like Docker. When the job is done, they're shut down.



Core Concepts

3. Jenkins **Node**

A Jenkins Node is a term that is used in Jenkins 2.0 to mean any system that can run Jenkins jobs. This is mainly used in Controllers and Agents and sometimes used instead of those terms. A node is a machine that Jenkins uses to run jobs like building and testing projects. If a node isn't performing well or its health drops below a certain threshold level, Jenkins will take that node offline to prevent any problems.

4. Jenkins **Job**

Jenkins jobs are used to perform the work in the Jenkins system. A Jenkins job is an automated job that you set up in Jenkins to do things like build, test or deploy your code. Instead of doing these steps manually every time, Jenkins can do them for you whenever there's a change in your code. This helps save time and reduces mistakes.

Core Concepts

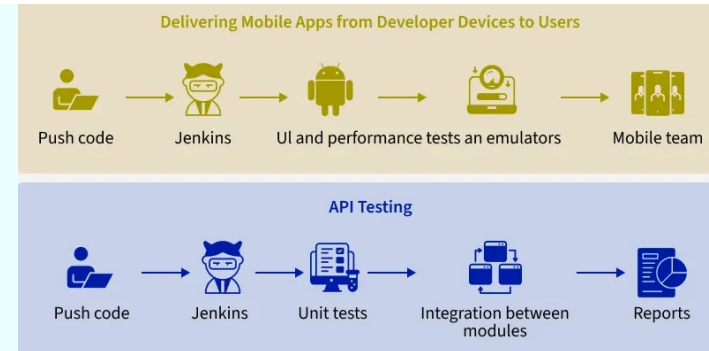
5. Jenkins **Plugins**

Jenkins provides **2000+** community contributed Jenkins plugins which developers can use for building, deploying and automating any project. You can connect to GitHub, Slack, AWS, Docker, and more—instantly amplifying Jenkins' power.. You can easily install and upgrade these available plugins easily from the Jenkins Dashboard.

6. Jenkins **Pipeline**

DevOps professionals mostly work with pipelines because pipelines can automate the processes like building, testing, and deploying the application. With the help of Continuous Integration / Continuous Deployment (CI/CD) Pipeline scripts we can automate the whole process which will increase productivity and save lots of time for the organization and can deliver quality applications to the end users.

Jenkins Pipeline



- **Code:** Developers write source code and manage it in Git repositories, which track all changes. Webhooks notify Jenkins on new commits, triggering build pipelines automatically.
- **Build:** Jenkins fetches the latest code and uses build tools like Maven, Gradle, or MSBuild via plugins to compile, package (JARs, WARs, containers), and run unit tests and code checks.
- **Test:** Jenkins coordinates different test types (UI, performance, security, compatibility) using frameworks like Selenium, providing detailed reports, logs, and metrics.
- **Signing/Security:** Builds undergo code signing, vulnerability scans, and manual approvals to enforce governance and security policies.
- **Deploy:** Jenkins deploys validated code to testing, staging, or production using containers or cloud platforms like AWS, Azure, Kubernetes, and Docker.
- **Inform:** Teams get automated email updates and dashboards showing pipeline status, logs, and reports.

Create Jenkins Job

First, you'll want to log into your Jenkins dashboard. This is like entering a workshop full of tools to help build your software. Once inside, click "New Item" to start a new project. Give your job a nice name - maybe after your favorite movie character or snack food.

Then picking your job type. There are a few options here that do different things:

- **Freestyle project** - Lets you run custom commands and scripts. Like following a recipe step-by-step.
- **Pipeline** - For stacking tasks together into an automated workflow. Kind of like an assembly line!
- **Multibranch Pipeline** - When you have code in multiple branches, and want to build from each. Like building several models of a toy from different molds.

Job Options

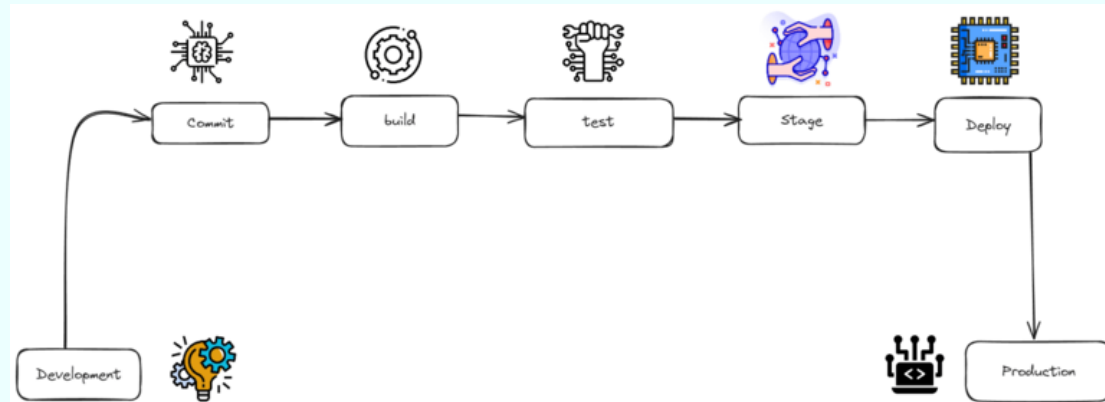
There are a few more types too, but these cover most use cases. Next, configure your job's settings. Here you can pick and choose what you want it to do - things like:

- Fetch code from version control
- Build and compile the code
- Run automated tests
- Deploy it somewhere after a successful build

The options are endless. Set up your job just as you need it. Finally, save your job and click "Build Now" to test it out. Watch your job execute each step and voila - you've successfully automated the build process!

Pipeline

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps {
        git 'https://github.com/org/repo'
      }
    }
    stage('Build') {
      steps {
        // compile code
        // save build artifacts to S3 bucket
      }
    }
    stage('Test') {
      steps {
        // download artifacts from S3
        // load test data into Postgres DB
        // run integration tests
        // continue only if all tests pass
      }
    }
    stage('Deploy') {
      steps {
        // download artifacts from S3
        // deploy to Kubernetes cluster
      }
    }
  }
}
```



- **Get latest code:** Uses the git step to check out code from a Git repository. Provides the URL to the repo as a parameter. Clones the latest commit to the local workspace. Ensures all subsequent steps have access to the freshest source code.
- **Build code:** Compiles the application source code into executables/packages. Saves the build artifacts like JARs, containers, etc to cloud storage. Uses S3 or an equivalent object store for storing the outputs. Makes the build outputs available for later stages.
- **Test code:** Fetches the build artifacts from the storage location. Loads test data and schemas into a Postgres database. Executes automated integration tests against the application. Continues only if all test cases pass as expected. Gates deployment on successfully passing tests.
- **Deploy code:** Downloads the vetted build artifacts for deployment. Deploys the application to a Kubernetes cluster. Pushes the containers to the target runtime environment. Makes the built and tested application available to users. This pipeline has discrete steps for each major phase of the CD process - repo checkout, build, test, and deploy. Each stage focuses on one concern and uses standard tools. This modular design enables extensibility and maintainability.

Declarative vs Scripted Pipeline

Declarative pipelines take a more structured, easy-to-visualize approach. The Jenkinsfile format allows you to lay out pipeline stages, such as build, test, and deploy, in a linear, sequential order. Each stage has a clean block defining what steps should occur within that stage. To me, this maps very cleanly to the progression of taking code from version control, through build, validation, and release processes. I can look at the Jenkinsfile like a flowchart and understand the logical order of events.

Scripted pipelines offer much more flexibility and customization capability, at the cost of increased complexity. Steps are defined procedurally using Groovy code encapsulated within methods like build(), test(), etc. The logic flow is harder for me to follow, as I have to trace through the method calls to see the overall sequence. The highly customizable nature of scripted pipelines enables much more sophisticated orchestration, but requires more Groovy programming expertise.

Declarative pipelines are best for simple, linear CI/CD workflows, while scripted pipelines offer greater flexibility and customization for complex scenarios.

Declarative vs Scripted Pipeline

```
pipeline {  
  agent any  
  stages {  
    stage('Build') {  
      steps {  
        // build steps  
      }  
    }  
    stage('Test') {  
      steps {  
        // test steps  
      }  
    }  
    stage('Deploy') {  
      steps {  
        // deploy steps  
      }  
    }  
  }  
}
```

```
node {  
  
  stage('Build') {  
    // build steps  
  }  
  
  stage('Test') {  
    // test steps  
  }  
  
  stage('Deploy') {  
    // deploy steps  
  }  
}
```

Pipeline Sample

```
node {  
    // Define the Maven tool used for the build process  
    def mavenHome = tool name: 'Maven 3.8.6'  
  
    // Configure build properties (e.g., log rotation and SCM polling)  
    properties([  
        buildDiscarder(logRotator((artifactDaysToKeepStr: '', artifactNumToKeepStr: '5', daysToKeepStr: '',  
numToKeepStr: '5'))),  
        pipelineTriggers([pollSCM('* * * * *')])  
    ])  
  
    // Checkout code from the Git repository  
    stage('Checkout Code') {  
        git branch: 'development', credentialsId: 'github-credentials-id', url: 'https://github.com/your-username/  
your-repository.git'  
    }  
  
    // Build the project using Maven  
    stage('Build') {  
        sh "${mavenHome}/bin/mvn clean package"  
    }  
  
    // Execute code quality analysis with SonarQube  
    stage('Execute SonarQube Analysis') {  
        sh "${mavenHome}/bin/mvn clean sonar:sonar"  
    }  
  
    // Upload the build artifact to Nexus repository  
    stage('Upload Build Artifact') {  
        sh "${mavenHome}/bin/mvn clean deploy"  
    }  
  
    // Deploy the application to Tomcat using SCP  
    stage('Deploy to Tomcat') {  
        sshagent(['your-ssh-credentials-id']) {  
            sh "scp -o StrictHostKeyChecking=no target/maven-web-application.war ec2-user@your-server:/opt/apache-  
tomcat-9.0.64/webapps/"  
        }  
    }  
}
```

Steps:

1. Login to Jenkins
2. Redirected to the Jenkins Dashboard
3. Create a New Project
4. Configure the Project Type
5. Configure the General Section
6. Set the build Triggers
7. Advanced Project Options
8. Configure the pipeline Section
9. Save the Pipeline and Run it
10. Monitor the Pipeline Execution
11. Review the Build Status

Example

```
Pipeline {  
  Agent {  
    Node {  
      Label 'SLAVE01'  
    }  
  }  
  
  Tools {  
    Maven 'maven3'  
  }  
  
  Options {  
    BuildDiscarder LogRotator(  
      DaysToKeepStr: '15',  
      NumToKeepStr: '10'  
    )  
  }  
  
  Environment {  
    APP_NAME = "CSTU_APP"  
    APP_ENV = "DEV"  
  }  
  
  Stages {  
    Stage('Cleanup Workspace') {  
      Steps {  
        CleanWs()  
        Sh 'echo "Cleaned Up Workspace for ${APP_NAME}"'  
      }  
    }  
  }  
}
```

```
    }  
  }  
  
  Stage('Code Checkout') {  
    Steps {  
      Checkout([  
        $class: 'GitSCM',  
        Branches: [[name: '*/master']],  
        UserRemoteConfigs: [[url: 'https://github.com/spring-projects/spring-  
petclinic.git']]  
      ])  
    }  
  }  
  
  Stage('Code Build') {  
    Steps {  
      Sh 'mvn install -Dmaven.test.skip=true'  
    }  
  }  
  
  Stage('Printing All Global Variables') {  
    Steps {  
      Sh 'env'  
    }  
  }  
}
```